

■ 2.3.8 Advanced Topic: The Complete Structure of Patterns

For most purposes, you should find it quite sufficient to use just the pattern objects we have discussed so far. In some advanced applications, however, you may also have to use other *Mathematica* pattern objects.

<code>_</code>	any expression
<code>x_</code>	any expression, given the name <i>x</i>
<code>x:pattern</code>	a pattern, given the name <i>x</i>
<code>pattern ? test</code>	a pattern that satisfies a test
<code>_h</code>	any expression with head <i>h</i>
<code>—</code>	any sequence of one or more expressions
<code>—</code>	any sequence of zero or more expressions
<code>x_:v</code>	an expression with default value <i>v</i>
<code>x_h:v</code>	an expression with head <i>h</i> and default value <i>v</i>
<code>x_.</code>	an expression with a globally-defined default value
<code>Optional[x_h]</code>	an expression that must have head <i>h</i> , and has a globally-defined default value
<code>pattern..</code>	a pattern repeated one or more times
<code>pattern...</code>	a pattern repeated zero or more times

Pattern objects in *Mathematica*.

Pattern Names

You will find the construction `x:pattern` useful when you give a pattern to define the structure of a part, but then want to give a single name to the whole part, so that, for example, you can use it on the right-hand side of a transformation rule.

This defines a function `f` whose argument can be *anything^anything*, with whole argument named `x`.

```
In[1]:= f[x:_.] := p[x]
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

x is the complete object a^b .

```
In[2]:= f[a^b]
```

```
Out[2]= p[ab]
```

Here is a more complicated case. The exponent is called n , and the complete object is called x .

```
In[3]:= g[x:_n_] := p[x, n]
```

Here x is a^b and n is b .

```
In[4]:= g[a^b]
```

```
Out[4]= p[ab, b]
```

Restricted Patterns

We have discussed two ways to restrict the expressions that a particular pattern can match. One is to use objects like `_Integer` that match only expressions with particular heads. Another, more general, approach that you can use in setting up transformation rules is to include a `/;` condition.

There is a third approach that *Mathematica* allows. You can restrict any pattern by using the construction *pattern* ? *test*. The pattern will be considered to match a particular expression only if the result of applying *test* to the expression is **True**.

When you set up transformation rules, it is usually best to make restrictions using `/;` conditions. However, when you have to give individual patterns, as in functions like **Cases** and **Position**, you cannot use `/;`, so ? is the only possibility.

In some sophisticated uses of transformation rules, it may also be important to make sure that *Mathematica* detects failure of pattern match before it has finished the whole pattern. In these cases, again, restrictions based on ? may be appropriate.

This defines a value for h when its argument satisfies the `NumberQ` test. You could have made a definition for *integer* x using `x_Integer`, but there are several different heads that an arbitrary *number* can have.

```
In[5]:= h[x_?NumberQ] := x^2
```

The definition applies to h with any numerical argument.

```
In[6]:= h[4.5] + h[3/2] + h[x]
```

```
Out[6]= 22.5 + h[x]
```

This defines a value for i with positive integer arguments.

```
In[7]:= i[x_Integer?Positive] := x^2
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This gives a definition for `j` when its argument is an integer `x` for which `Function[v, v > 2][x]` is `True`. In this case, a definition like `j[x_Integer] := x^2 /; x > 2` would probably be better.

```
In[8]:= j[x_Integer?Function[v, v > 2]] := x^2
```

<code>Positive[x]</code>	strictly greater than zero
<code>NonNegative[x]</code>	greater than or equal to zero
<code>Negative[x]</code>	less than zero
<code>NumberQ[x]</code>	a number of any kind

Functions for testing properties of expressions.

You can always explicitly give a pure function as the test to use in a `?` construction. For some common tests, however, there are built-in *Mathematica* functions. Thus, for example, you can use `x?NumberQ` to stand for any number, and `x_Integer?NonNegative` to stand for any non-negative integer. Note that a `?` test is assumed to have failed if it does not explicitly come back `True`.

Repetitive Patterns

The final element of patterns that we need to discuss is the “repetition operator”. The object `_` represents any sequence of expressions. You can use `pattern..` to represent sequences of expressions, in which each expression matches a particular pattern.

This gives a definition for `j` when it has a sequence of arguments, all of the form `k[anything]`. The whole sequence of arguments is named `x`. (Using the pattern `x_k` would not guarantee that each `k` had exactly one argument.)

```
In[9]:= j[x:k[_]..] := jp[x, x]
```

The pattern matches in this case.

```
In[10]:= j[k[1], k[2], k[3]]
Out[10]= jp[k[1], k[2], k[3], k[1], k[2], k[3]]
```

It does not match in this case.

```
In[11]:= j[k[1], 2, k[3]]
Out[11]= j[k[1], 2, k[3]]
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

The arguments of `s` can be any sequence of objects of the form `m[anything, n]`, where `n` must be the same for all the arguments.

```
In[12]:= s[x:m[_ ,n_]..] := sp[x, x]
```

This matches.

```
In[13]:= s[m[1,1], m[2,1], m[4,1]]
Out[13]= sp[m[1, 1], m[2, 1], m[4, 1], m[1, 1], m[2, 1],
           m[4, 1]]
```

This does not.

```
In[14]:= s[m[1,1], m[2,2]]
Out[14]= s[m[1, 1], m[2, 2]]
```